

GameTrainer — Milestone M0

Setup and the Loop: Proving the Reinforcement-Learning Foundation

Author: Phillip • Date: July 7, 2026

Report type: milestone retrospective (M0)

Abstract

Milestone 0 (M0) is the first step of GameTrainer's reinforcement-learning (RL) foundation: it stands up the project skeleton and proves that the fundamental observe–act–reward loop executes end-to-end, using a borrowed, well-known environment (CartPole) so that no risky new code is on the critical path. The milestone's "done when" criterion — a script running CartPole with random actions for 100 steps without crashing — was met, and the run produced a documented random-action baseline of approximately 22 reward per episode. That number matters beyond M0: it is the yardstick Milestone 1 (M1) must beat to prove that learning is actually happening. M0 also quietly wires in several pieces of infrastructure — an input-controller seam, a hardware detector, a logger, and a text-menu launcher — that every later milestone reuses unchanged.

1. Project Context & Mental Model

GameTrainer is a personal research and learning project that trains an AI agent to play games. Its real subject is not any single game but an **architecture**: a clean, swappable link between any game world and any AI "brain." The project is built in strict milestone order, M0 through M6, starting from the simplest possible setup and working upward. A real commercial game, Stardew Valley, is the final step of the roadmap, not the first — M0 begins with almost nothing at stake.

1.1 Shared Vocabulary

The project uses a consistent set of terms throughout its documentation:

- **Ground** — the game or environment. Its job is to (1) show what's happening (an **observation**) and (2) grade the agent (a **reward**).
- **AI** — the player, itself three parts: **eyes** (a Vision Transformer, added later, not in M0), a **brain** (a PPO reinforcement-learning agent, arriving in M1), and **hands** (an input controller that presses keys).
- **Link** — the **Gymnasium contract**, a standard software "socket" every environment exposes: `reset()` (start a fresh attempt) and `step(action)` (advance one tick and report back an observation, a reward, and whether the attempt ended).
- The core loop, always: **observe** → **act** → **reward** → **repeat**.
- **Design principle** — **build vs. borrow**. The project builds the Ground and the Link, but borrows the Brain (PPO, from the Stable-Baselines3 library, or "SB3") and, later, the Eyes backbone (a pretrained ViT). Borrowing is only possible because every Ground speaks the same Gymnasium contract — that swappability is the entire point of the project.

1.2 Why This Foundation Exists

GameTrainer began in 2025 as a different kind of program: a C++ / Windows-API effort that involved reading game memory directly and hand-authoring rule-engine and decision-tree logic. In early 2026 the project pivoted to a reinforcement-learning approach — learning to play purely from the outside, pixels in and key-presses out, using a borrowed PPO brain and a pretrained vision backbone, rather than reading memory or scripting rules. M0 is the first milestone of that fresh RL foundation, deliberately kept tiny so the loop itself can be trusted before anything ambitious is attempted.

1.3 The Roadmap

#	Goal	"Done when..."
M0 ✓	Setup	Repo + virtual environment created, libraries installed, a script runs CartPole with random actions for 100 steps without crashing.
M1	Borrow the brain	PPO trains on CartPole through our runner; average reward clearly rises vs. the random baseline.
M2	Build our own Ground	A GridWorld env obeys the Gymnasium contract; a random agent runs, then PPO learns to reach the goal.
M3	Add the Eyes	A ViT feeds a picture of the world to PPO; the agent learns from pixels.
M4	Make it swappable	Profiles + reward calculators exist; switching environments is config-only.
M5	Add the Hands	Real key presses drive a small real game window end-to-end.
M6	(Stretch) Stardew	Stardew becomes "just another profile."

Table 1. Full M0–M6 roadmap. M0 is complete as of this report.

Timeline. The broader project started mid-2025 in its C++/memory-reading era and pivoted to RL in early 2026. M0 (Setup / the Loop) was built roughly June 17–18, 2026. Both the run and the tooling executed entirely on CPU — no GPU is required at this stage.

2. Milestone 0 — Setup & the Loop

Objective. Stand up the project's skeleton and prove the fundamental RL loop executes end-to-end, using a borrowed environment so nothing risky is built yet.

"Done when..." Repo and virtual environment created, libraries installed, and a script runs CartPole with random actions for 100 steps without crashing.

2.1 What CartPole Is

CartPole is a classic control benchmark environment that ships with Gymnasium: a cart slides left and right on a track with a pole balanced upright on a hinge, and the agent nudges the cart to keep the pole from falling.

- **Observation** — 4 numbers (float): cart position, cart velocity, pole angle, pole angular velocity. Not an image — this matters for M1's choice of policy network.
- **Action space** — Discrete(2): 0 = push left, 1 = push right.
- **Reward** — +1 for every timestep the pole stays up.
- **Episode ends** — “terminated” if the pole falls past $\pm 12^\circ$ or the cart leaves the track; “truncated” at a 500-step cap, so the maximum possible score per episode is 500.

2.2 What We Built

M0 consists of three live pieces plus a layer of supporting infrastructure that every later milestone reuses.

1. Borrowed CartPole environment. Created with `gym.make("CartPole-v1")`. No environment code is authored here — we borrow a known-good Ground purely to test the plumbing around it.

2. Random-action runner — `scripts/run_cartpole.py`. It calls `env.reset()`, then loops 100 steps. Each step samples a random legal action via `env.action_space.sample()`, calls `env.step(action)`, accumulates reward, and re-resets whenever an episode ends. It prints periodic status and, at the end, the mean reward per episode — this is the **M0 random baseline** that M1 must beat.

3. NullInput input stub — `src/gametrainer/input.py`. This is the “hands” seam, wired in early, before it is needed. `NullInput` is a subclass of the `InputController` interface whose every method is a no-op. `CartPole` is programmatic (the environment moves the cart directly), so no real key press is required — but the runner still calls `hands.tap_key(action)` on every step so the eyes→brain→hands shape is visible from day one. `InputController` is designed to eventually call a compiled C++ input extension (`c lib`) for reliable real-game key presses; that extension is not built in M0–M2, and the code falls back to a harmless mock so early milestones run with no C++ compiler required. This is deliberate scope control.

4. Supporting infrastructure, built alongside and reused by every later milestone: a hardware/accelerator detector (`src/gametrainer/hardware.py`) that picks the best available device — CUDA (NVIDIA GPU) → MPS (Apple GPU) → CPU — without hard-requiring a GPU, plus a small `Logger` and a retro text-menu (TUI) launcher.

2.3 Why We Built It This Way

- **Borrow before build.** Using `CartPole` first isolates one variable: does the environment, tooling, and loop actually work? If something breaks, it's the project's plumbing at fault, not a bug in a hand-written Ground.
- **Wire the input seam early.** Adding `NullInput` before any real game exists proves the swappable “hands” socket is real. Later, in M5, a `KeyboardInput` drops into the same slot with zero changes to the loop — the socket is designed before anything is plugged into it.
- **Measure a baseline.** A random policy gives a number to beat. Without a baseline, “it learned” is an opinion; with one, it's a measurement.

2.4 Result & Verification

The loop ran for 100 random steps without crashing — satisfying the “done when...” criterion — and the runner reported a random-action baseline of ≈ 22 reward per episode. A random policy keeps the pole up for roughly

20–25 steps before it falls; the codebase uses 22 as the conservative, documented baseline figure.

2.5 What M0 Lends to Later

Everything M0 established — the Gymnasium contract, the input seam, hardware detection, the logger/TUI, and a measured baseline — is reused unchanged in M1, M2, and beyond. M0 is the socket; every later milestone plugs into it.

3. Architecture at M0

At M0 there are only three live pieces: the borrowed CartPole environment, the random-action runner, and the NullInput seam. The diagram below is a simplified rendering of the project's UML class diagram for this milestone. Solid arrows with hollow heads denote interface implementation; dashed arrows denote a run-time dependency.

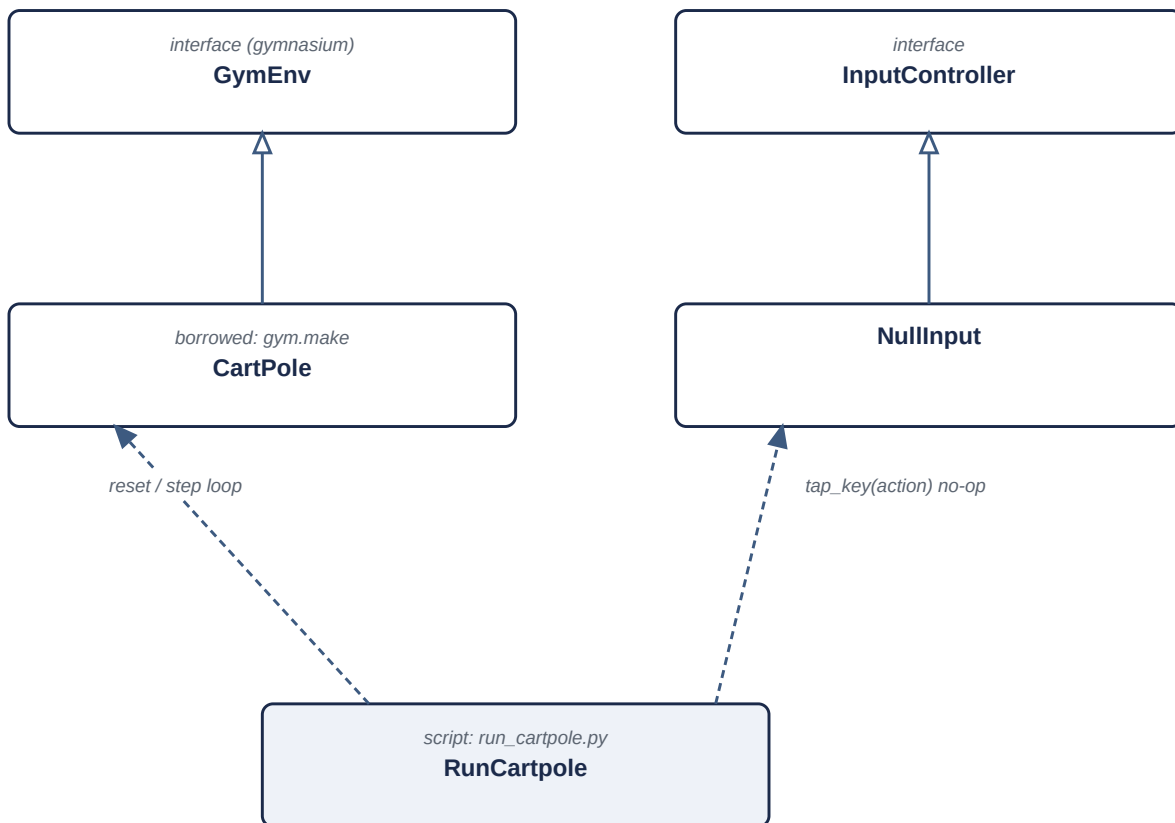


Figure 1. M0 architecture — RunCartpole drives CartPole through the Gymnasium contract and calls NullInput on every step, even though the no-op hands do nothing yet.

Both CartPole and NullInput sit behind stable interfaces (GymEnv and InputController respectively). Nothing about RunCartpole's logic depends on CartPole specifically being CartPole, nor on NullInput specifically doing nothing — which is precisely what allows M1 to swap in a learning agent without touching this diagram's shape.

4. Scope Discipline

M0 deliberately excludes a long list of capability, each mapped to the milestone where it will eventually arrive:

Excluded at M0	Arrives at
No eyes / no ViT / no pixels — the agent (from M1 onward) learns from CartPole's 4 numbers, not images.	Vision is M3
No Ground of our own — CartPole is borrowed.	Our first authored environment, GridWorld, is M2
No real key presses — the hands seam is a no-op stub.	Real input is M5
No reward calculator or profile abstraction — nothing to configure yet.	Config-driven swappability is M4
No GPU dependency — CPU is sufficient.	GPU only matters once the ViT phase begins

This restraint is a feature, not a limitation: keeping each milestone tiny is what makes the loop and the contract trustworthy before complexity is added on top of them.

5. Reflections & Lessons

- **Baseline before brain.** A number to beat turns “it learned” from an impression into a measurement.
- **Design the seam before you need it.** NullInput cost almost nothing to add at M0 and will save a refactor at M5.
- **Isolate one variable by borrowing.** Testing plumbing against a known-good environment means any failure is unambiguously the project's own code.
- **Small scope is verifiable scope.** A 100-step, no-crash criterion is trivial to check and leaves no room for a false sense of progress.

6. Next Steps

M1 replaces the random action-picker with a learning PPO agent, borrowed from Stable-Baselines3, and proves it beats the M0 baseline without changing the Gymnasium contract at all. See the companion report, *GameTrainer — Milestone M1: Borrow the Brain*, for that work.

Appendix A — Reproduce It

```
pip install -e "[rl]"
python scripts/run_cartpole.py
```

Appendix B — File Manifest

File	Role
scripts/run_cartpole.py	Random-action runner; reports the M0 baseline.
src/gametrainer/input.py	InputController interface and the NullInput no-op implementation.
src/gametrainer/hardware.py	Accelerator detector (CUDA → MPS → CPU).

Appendix C — Glossary

Ground

The game or environment being played; produces observations and rewards.

AI

The player: eyes (vision), brain (decision-making), and hands (input).

Link

The Gymnasium contract (reset/step) that lets any Ground and any AI connect through the same socket.

Observation

What the Ground reports back about the current state of the world.

Reward

The numeric grade the Ground gives the agent for a given step.

Gymnasium

The standard Python RL environment interface / contract used throughout the project.

CartPole

A classic Gymnasium control benchmark: balance a pole on a moving cart.

Episode

One attempt, from reset() to termination or truncation.

Baseline

A reference performance number (here, a random policy's average reward) used to judge whether learning is really happening.

InputController

The interface through which the AI's "hands" send actions to a real or simulated game.

NullInput

A no-op implementation of InputController used before real key presses are needed.